

Полная переписка

Экспорт текущего диалога в PDF без сокращений из доступного содержимого чата.

1. Пользователь

Теперь вопрос про RAG, как она у меня сейчас работает. И какие варианты есть её улучшить и в целом какие есть методы в целом сейчас? Я так понимаю, что данный RAG самописный и вряд ли будет работать корректно вдолгую.

В разработке от chainlit планирую переход в open web ui, где я буду подключать RAG систему по типу qdrant или milvus, плюс туда подключать в систему эмбеддера и тд. В общем, полноценную систему. В чате он скорее всего будет работать, но вопрос сессии, как будет дело обходиться с ними, когда вопрос будет по самому документу и как это лучше реализовать.

Про open web ui. Там имеется поддержка оркестрации со стороны open web ui, а модули отдельные для работы rag

2. Ассистент

Да, по коду у тебя сейчас RAG действительно скорее самописный гибридный контур, а не “полноценная production RAG-платформа”. Для прототипа это нормально, но надолго в таком виде я бы его не оставлял. По текущим файлам видно, что у тебя уже есть хорошая база: роутинг, score, session docs, knowledge-base слой, chunker, hybrid retrieval и эвристики уверенности. Но узкие места тоже уже видны.

Как у тебя RAG работает сейчас

Сейчас у тебя есть два основных контура.

Первый — session RAG: документы хранятся в сессии/чате, режутся LegalDocumentChunker, затем индексируются прямо в AdaptiveRAGPipeline, который использует HybridRetriever с BM25 + dense retrieval + RRF. В Chainlit индекс собирается заново по активным документам, кэшируется по набору active_doc_ids, а сам pipeline кладётся в user_session. Это означает, что RAG завязан на состояние текущей сессии и выбранный набор активных документов.

Второй — knowledge-base RAG: есть SQLite-хранилище с таблицами источников, чанков и эмбеддингов. При ingestion документ регистрируется как source, чанки сохраняются, эмбеддинги могут быть предвычислены и записаны в SQLite. При retrieval собираются записи из knowledge base и, при необходимости, поверх них накладываются session-документы как overlay. Затем результаты по score объединяются и скоряются эвристически.

На уровне retrieval у тебя есть несколько режимов:

- simple: BM25 + dense → RRF → top-K → контекст.
- corrective: intent classifier → hybrid search → z-score grading → при плохом результате Rocchio expansion и повторный retrieval.
- agentic: по факту это не полноценный агент, а iterative corrective path.
- multi-agent: пока просто fallback в agentic.

Чанкинг тоже не тупое: он секционно-ориентированный, заточен под юридические/структурированные документы, старается уважать заголовки, нумерацию и не разрезать внутри нумерованных пунктов. Это хороший плюс для твоего домена.

Ещё важный слой — orchestration. У тебя запрос сначала классифицируется: это общий чат, вопрос по документу, summary, compare, equipment analysis и т.д. Если система считает, что нужен RAG и есть документы/knowledge base, она уводит запрос в document_question или смежный workflow. То есть RAG у тебя не автономный сервис, а часть общего decision layer.

Почему это “вряд ли будет корректно работать вдолгую”

Главная проблема не в том, что оно самописное. Главная проблема в том, что сейчас это локально-сессионная архитектура с эвристическим retrieval-контуром, а не устойчивый retrieval stack.

Самые заметные ограничения такие.

1. Session-bound индекс.

В Chainlit индекс строится по активным документам в сессии и кэшируется в session state. Это удобно для прототипа, но плохо для долгой жизни: другой воркер, рестарт, миграция UI, несколько инстансов — и логика становится хрупкой.

2. SQLite как knowledge store.

SQLite нормален как метаданные/локальная база, но не как долгосрочная основа для масштабируемого retrieval слоя с конкурентной записью, фильтрацией, tenant isolation и большим объёмом индексов. У тебя retrieval из KB фактически поднимает чанки и эмбединги из SQLite и дальше сам прогоняет поиск через HybridRetriever, то есть vector DB как сервиса тут ещё нет.

3. Нет полноценного reranker слоя.

Сейчас есть grading по z-score и Rocchio expansion, но это не то же самое, что cross-encoder / ColBERT / late interaction reranking. На реальных корпусах именно reranking часто даёт самый заметный прирост точности. Текущий стек это частично имитирует, но не заменяет.

4. Интент-классификация завязана на centroid examples.

Для дешёвого рантайма это ок, но со временем такие классификаторы начинают давать нестабильность на пограничных запросах, особенно когда появляется больше сценариев. У тебя даже в коде есть threshold/margin логика, чтобы компенсировать неуверенность.

5. “Agentic” пока не agentic.

В коде прямо видно, что agentic — это не planner + tool-use runtime, а iterative corrective path. То есть ожидание “умного многошагового поиска” пока не совпадает с реальной реализацией.

6. Эвристическая confidence / evidence validation.

Это лучше, чем ничего, и для grounded QA это полезно, но при росте типов документов, языков и форматов такие эвристики обычно начинают трещать. Они годятся как guardrail, но не как главный механизм качества.

Какие варианты улучшения есть именно для твоей текущей архитектуры

Если идти эволюционно, а не всё переписывать сразу, я бы делал так.

Сначала перевести RAG из “индекса в сессии” в двухконтурную модель:

- session overlay для только что загруженных файлов;
- persistent KB в отдельной vector DB для всего долговременного корпуса.

У тебя это уже частично задумано: source_scope_summary умеет различать session, knowledge_base и knowledge_base+session_overlay. Значит архитектурно ты уже почти туда пришёл. Надо просто довести это до конца.

Дальше я бы заменил текущий retrieval backend на внешний vector DB и оставил orchestration как отдельный слой. Для Open WebUI это как раз естественная модель: UI отдельно, knowledge/retrieval отдельно, pipe/pipeline отдельно. Open WebUI поддерживает Knowledge, Pipelines, Functions/Tools и несколько vector DB, включая Qdrant и Milvus. Для multi-worker и multi-replica их как раз рекомендуют вместо локального SQLite-backed Chroma.

Что делать с сессией, когда вопрос “по самому документу”

Лучший вариант — не пытаться хранить истинный retrieval state только в UI-сессии.

Правильнее разделить три сущности:

- chat/session state: история, активный режим, выбранные документы;
- document state: сами документы, их версии, метаданные;
- retrieval state: коллекция, tenant, namespace, filters, индексы.

Тогда логика такая:

1. Пользователь загрузил документ.
2. Документ получил document_id, file_id, hash, tenant_id, thread_id.
3. Документ ушёл в ingestion pipeline.
4. Из чата ты хранишь только ссылки: какие документы сейчас “attached/active”.
5. При вопросе по документу ты не берёшь индекс из оперативки, а делаешь retrieval по фильтру:
tenant_id + thread_id + active_doc_ids.

Так ты получаешь:

- восстановление после рестарта;
- отсутствие жёсткой привязки к одному воркеру;
- возможность масштабировать Open WebUI;
- понятное поведение “этот вопрос относится к этому документу”.

Что лучше: Qdrant или Milvus

Для твоего кейса я бы сформулировал так.

Qdrant лучше, если тебе нужны:

- более простой и понятный старт;
- payload filters;
- удобная multitenancy;
- хороший DX для hybrid retrieval и последующего reranking;
- меньше инфраструктурной тяжести на старте.

Milvus лучше, если тебе нужны:

- очень большой масштаб;
- multi-vector / dense+sparse / сложные ANN-конфигурации;
- partition-ориентированная изоляция;
- большой запас на рост multi-tenant нагрузки.

Для твоего перехода с Chainlit в Open WebUI я бы скорее выбрал Qdrant как первый production шаг, если ты не ожидаешь сразу очень большой multi-tenant enterprise-scale нагрузки. Он проще как “первый серьёзный backend”. Milvus я бы брал, если ты уже знаешь, что у тебя будет большой объём, тяжёлый distributed режим или сложная multi-vector схема.

Что Open WebUI умеет по этой части

Open WebUI уже сам по себе умеет RAG/Knowledge, модели можно привязывать к knowledge base, а для расширения логики есть Pipes, Pipelines, Tools и Functions. Для масштабирования документация Open WebUI прямо предупреждает, что локальный SQLite-backed vector backend не подходит для multi-worker / multi-replica сценариев, и рекомендует внешние vector DB вроде Qdrant или Milvus. Также у него есть Knowledge-режим, возможность привязки knowledge к модели, а при native function calling knowledge может использоваться как tool-вызовы, а не только как автоматическая подмешка контекста.

То есть на твой вопрос “там имеется поддержка оркестрации со стороны Open WebUI, а модули отдельные для работы RAG?” — ответ: да, но лучше воспринимать Open WebUI не как готовый replacement твоего orchestration core, а как UI + knowledge/workflow platform. Бизнес-логику оркестрации лучше оставить у себя в pipe/pipeline/backend-сервисе, а Open WebUI использовать как оболочку и операторский интерфейс.

Какие методы RAG вообще сейчас актуальны

Если смотреть шире, сегодня обычно используют не один “RAG”, а стек техник.

Базовый уровень:

- good chunking;
- dense retrieval;
- sparse retrieval/BM25;
- hybrid fusion;
- metadata filtering.

Следующий уровень:

- query expansion;
- reranking;
- late interaction / ColBERT-подобные схемы;
- multi-vector retrieval.

Ещё выше:

- corrective / iterative RAG;
- self-reflection / self-RAG-подходы;
- graph-based retrieval / GraphRAG для сложных связей;
- table-aware retrieval для табличных/гетерогенных данных.

Для твоего кейса я бы не шёл сразу в GraphRAG. У тебя сначала даст больше пользы вот такая последовательность:

1. хороший persistent vector store;
2. hybrid retrieval;
3. metadata filters;
4. reranker;
5. normal citations/evidence layer;
6. only then iterative/agentic retrieval;
7. GraphRAG — только если реально есть графовые зависимости между сущностями.

Что бы я предложил как целевую архитектуру для тебя

Самая здравая схема на переходе Chainlit → Open WebUI такая:

UI

Open WebUI

Orchestration layer

твой backend / pipe / pipeline:

- intent routing
- scope resolution
- session/document binding
- answer synthesis
- citations/confidence

Ingestion layer

отдельный ingestion worker:

- parse
- normalize
- chunk
- embed
- index
- versioning

Retrieval layer

Qdrant или Milvus:

- dense + sparse
- filters
- rerank candidate set
- tenant/thread/document scoping

Metadata layer

PostgreSQL:

- users
- chats
- threads
- documents
- versions
- active attachments

- collection bindings

Optional cache

Redis:

- short-lived retrieval cache
- background ingestion status
- session hints

Тогда “вопрос по самому документу” решается не через память текущего UI-процесса, а через metadata + vector filters.

Мой честный вывод по твоей текущей реализации

У тебя сейчас не “плохой” RAG. У тебя нормальный инженерный прототип с уже продуманными кусками:

- гибридный retrieval;
- чанкинг под домен;
- разделение session и KB;
- routing;
- эвристики groundedness.

Но это всё ещё prototype-grade architecture, потому что:

- индекс живёт слишком близко к сессии;
- persistent retrieval backend ещё не production-class;
- reranking пока слабый;
- agentic слой пока условный;
- масштабирование и multi-instance устойчивость ещё не доведены.

Так что направление у тебя правильное: Open WebUI + внешний vector DB + отдельный embedder/ingestion + сохранение собственного orchestration слоя. Это как раз логичный следующий шаг.

Если хочешь, следующим сообщением я разложу тебе это уже в виде конкретной целевой схемы реализации для Open WebUI: компоненты, потоки данных, storage model, session model и recommended stack по шагам.

3. Пользователь

<https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2/>

<https://huggingface.co/Qwen/Qwen3-Reranker-4B>

[Alibaba-NLP/gte-reranker-modernbert-base](https://huggingface.co/Alibaba-NLP/gte-reranker-modernbert-base)

<https://huggingface.co/facebook/bart-large-cnn/>

<https://huggingface.co/facebook/bart-large-mnli>

[ahotrod/electra_large_discriminator_squad2_512](https://huggingface.co/ahotrod/electra_large_discriminator_squad2_512)

[BAAI/bge-reranker-v2-m3](https://huggingface.co/BAAI/bge-reranker-v2-m3)

Что-нибудь из этого под это подразумевается?

4. Ассистент

Да — под “reranker” из твоего списка напрямую подходят именно эти модели:

- cross-encoder/ms-marco-MiniLM-L6-v2 — классический cross-encoder reranker для passage ranking / retrieve-then-rerank. Он обучен на MS MARCO и прямо описан как модель для IR, которая сортирует уже найденные passages.
- Qwen/Qwen3-Reranker-4B — современный text reranking-модель, многоязычная, с контекстом до 32k, но уже заметно тяжелее по ресурсам.
- Alibaba-NLP/gte-reranker-modernbert-base — тоже text reranker, англоязычный, 149M, до 8192 токенов; выглядит как очень практичный вариант по балансу качества и веса.
- BAAI/bge-reranker-v2-m3 — тоже именно reranker, причём multilingual и позиционируется как лёгкий в деплое и быстрый.

А вот это — не reranker:

- facebook/bart-large-cnn — модель для summarization. Её место — постобработка, например “сжать найденный контекст”, но не rerank кандидатов.
- facebook/bart-large-mnli — zero-shot classification / NLI. Её можно использовать для intent classification, topic routing, label selection, но не как нормальный retrieval reranker.
- ahotrod/electra_large_discriminator_squad2_512 — question answering reader-модель. Это extractive QA по уже данному контексту, а не reranker.

Если перевести это на твой стек, то логика такая:

Для RAG-пайплайна

1. сначала embeddings + vector DB,
2. затем top-k retrieval,
3. потом reranker,
4. и только после этого LLM-генерация ответа.

То есть из твоего списка для шага №3 подходят только:

ms-marco-MiniLM-L6-v2, Qwen3-Reranker-4B, gte-reranker-modernbert-base, bge-reranker-v2-m3.

Практически я бы тебе рекомендовал так:

- если нужен быстрый и дешёвый старт — cross-encoder/ms-marco-MiniLM-L6-v2; он очень распространён и быстрый, но в основном англоязычный.
- если нужен хороший современный англоязычный reranker под документы — Alibaba-NLP/gte-reranker-modernbert-base.
- если важна мультиязычность, включая русский — смотреть прежде всего на BAAI/bge-reranker-v2-m3 или Qwen/Qwen3-Reranker-4B; у Qwen больше мощность и длинный контекст, но он тяжелее, у BGE обычно проще для практического прод-внедрения.

Под твой сценарий перехода в Open WebUI + Qdrant/Milvus я бы это видел так:

- embedder отдельно
- vector DB отдельно
- reranker отдельно
- LLM отдельно
- summarizer/QA classifier — опционально как вспомогательные модули

То есть bart-large-cnn, bart-large-mnli, electra...squad2 — это уже не “ядро rerank слоя”, а дополнительные сервисные головы для summary / routing / extractive QA.

Если выбирать совсем прикладно под твой кейс “сейчас и без перегруза”, я бы смотрел в таком порядке:

bge-reranker-v2-m3 → gte-reranker-modernbert-base → Qwen3-Reranker-4B,
а ms-marco-MiniLM-L6-v2 держал бы как лёгкий baseline.

Могу сразу после этого собрать тебе небольшую таблицу: что из них лучше для RU/EN, по ресурсам, скорости и куда ставить в пайплайне Open WebUI.

5. Пользователь

Так, другой вопрос, если я хочу плавно переписать не на самописный rag, а полноценно на продукты, которые сейчас имеются

6. Ассистент

Да, это как раз правильный разворот мысли: не “допиливать самописный RAG до бесконечности”, а перейти на продуктовый стек, где ты собираешь систему из уже готовых компонентов: UI, ingestion, vector DB, retrieval/rerank, orchestration. Open WebUI для этого подходит как оболочка, но не как единственный “движок всего”. У него есть встроенный RAG/Knowledge, поддержка нескольких vector DB, а также Pipes/Functions для кастомной логики. При этом сами docs Open WebUI отдельно предупреждают, что дефолтный локальный Chroma на SQLite не подходит для multi-worker / multi-replica, и для масштабирования нужно уходить на

внешние vector DB вроде Qdrant, Milvus или PGVector.

Если говорить практично, у тебя сейчас есть три реальных пути.

Путь 1: Open WebUI как почти готовый продукт, минимум своего кода.

Это самый мягкий переход. Ты используешь Open WebUI как UI + Knowledge, подключаешь внешний vector DB, нормальный embedder и по возможности внешний parser. Такой вариант быстрее всего запускается, но ты будешь ограничен тем, как Open WebUI организует knowledge flow. Для простого “чат по документам / KB / прикрепленные файлы” этого часто хватает. Для сложного routing, гибридного retrieval, хитрой session logic и спецправил по документам — быстро упрёшься в кастомизацию.

Путь 2: Open WebUI как фронт, а RAG вынести в отдельный pipeline/backend.

Это обычно лучший компромисс. Open WebUI остаётся интерфейсом и точкой доступа для пользователей, а вся “настоящая” логика живёт в отдельном pipe/pipeline-сервисе: ingestion, filters, hybrid retrieval, rerank, citations, document-scoping, session overlay. Open WebUI Pipes как раз для этого и сделаны: они могут вызывать внешние сервисы и показываться в UI как отдельные “модели/агенты”. Для твоего кейса это выглядит самым здоровым вариантом.

Путь 3: вообще перейти на более “RAG-first” фреймворк как основу.

Здесь самые логичные кандидаты — LlamaIndex и Haystack. LlamaIndex силён в ingestion, retrieval abstractions, индексах, node postprocessing, rerank и интеграциях с vector stores; у него есть и managed-вариант через LlamaCloud с ingestion/retrieval API. Haystack позиционируется как framework для production-ready RAG/agents/search, с явными retrievers, rankers и pipeline-компонентами. Если тебе нужен именно “RAG как система”, а не просто web chat с документами, эти фреймворки обычно лучше подходят, чем пытаться строить всё на одном лишь UI-продукте.

Если переводить это в архитектурные рекомендации, то я бы советовал не искать “один продукт, который решит всё”, а разделить стек на слои.

Что оставить продуктовым слоем:

- Open WebUI — интерфейс, пользователи, чаты, модели, доступ;
- Qdrant или Milvus — vector store;
- reranker — отдельный сервис/модель;
- parser/ingestion — отдельный сервис или managed ingestion;
- PostgreSQL — метаданные, сессии, документы, bindings.

И уже поверх этого либо LlamaIndex/Haystack, либо свой тонкий orchestration слой. Такая сборка обычно устойчивее, чем монолитный “самописный RAG внутри UI”.

По векторной базе выбор сейчас у тебя реально между Qdrant и Milvus.

Qdrant хорош, если нужен более простой старт, хорошие payload filters, hybrid queries и понятная multitenancy-модель. У него есть официальные материалы по hybrid search с dense+sparse и по hybrid queries через RRF. Для “перевести текущую систему на продуктовый backend без лишней тяжести” Qdrant выглядит очень сильным первым кандидатом.

Milvus сильнее выглядит там, где тебе важны multi-vector схемы, встроенные варианты reranking в hybrid search и более тяжёлый масштаб. В документации у него прямо есть multi-vector hybrid search, filtered search и ranker strategies вроде WeightedRanker и RRFRanker. Это мощно, но обычно инфраструктурно чуть тяжелее, чем Qdrant.

Если говорить совсем предметно под твою ситуацию, я бы рекомендовал так:

Если тебе нужен плавный переход с минимальным риском — Open WebUI + Qdrant + отдельный reranker + свой pipeline service.

Это даст тебе:

- не ломать UX,
- не переписывать всё сразу,
- убрать session-bound индекс,
- уйти от SQLite/локального Chroma,
- оставить возможность постепенно переносить логику из Chainlit.

Если хочешь максимально “продуктовый RAG backend”, а не просто UI-платформу — Open WebUI спереди, а сзади LlamaIndex или Haystack как orchestration/RAG framework. LlamaIndex удобен, если тебе важны ingestion pipelines, managed retrieval, постпроцессоры и большой выбор retrieval abstractions. Haystack удобен, если хочется более явных production pipelines с retriever → ranker → generator и понятной композицией компонентов.

Если хочешь вообще меньше собственной инфраструктуры — смотреть в managed ingestion/retrieval, например LlamaCloud, где есть managed ingestion API и managed retrieval API. Но это уже не self-hosted-first история, и тут нужно отдельно смотреть требования по приватности, бюджету и vendor lock-in.

Самый важный вывод: переписывать “на продукты” стоит не как замену одной библиотеки на другую, а как переход от самописного монолита к слоистой архитектуре.

То есть не “вместо моего RAG теперь будет Open WebUI”, а:

- Open WebUI = UI и точка входа,
- vector DB = поиск,
- reranker = точность,
- ingestion = загрузка и обновление документов,
- orchestration = правила, маршрутизация, session/doc scope.

Тогда у тебя и сессии, и вопросы “по конкретному документу”, и постоянная KB будут работать предсказуемо. Это особенно важно, потому что сами docs Open WebUI прямо указывают на необходимость внешних БД и shared storage в масштабируемом деплое.

Моя рекомендация по порядку миграции такая:

1. Сначала оставить текущую логику routing, но вынести retrieval в внешний vector DB.
2. Потом убрать session-only индекс и перейти на document/thread filters.
3. Потом добавить reranker.
4. Потом заменить самописные ingestion/retrieval куски на LlamaIndex или Haystack там, где это реально упрощает код.
5. И только потом решать, нужен ли тебе полностью managed стек.

Если сказать совсем прямо:

для тебя сейчас наиболее здравый вариант — не “переписать всё на продукт”, а “собрать продуктовый стек вокруг Open WebUI”.

То есть Open WebUI + Qdrant + reranker + свой pipeline, а затем по мере стабилизации заменить часть своего orchestration на LlamaIndex или Haystack, если это даст выигрыш в поддержке.

Могу дальше разложить это уже в виде конкретных 2–3 migration-сценариев: “дёшево и быстро”, “сбалансированно”, “почти enterprise”.

7. Пользователь

Итак, на основе всей нашей с тобой переписки сделай мне конкретный подробный технический план в виде чекбоксов (+ контекст с объяснением), то есть как тз для разработчика по реализации и переходу к open web ui и всей этой полноценной реализацией под то, что у нас сейчас имеется в chainlit и по роутингу, чтобы переход был плавный. Формат задачи в md коде. Можешь файлом.

8. Ассистент

Готово. Я собрал ТЗ в markdown-файл:

[Скачать openwebui_migration_tz.md](sandbox:/mnt/data/openwebui_migration_tz.md)

Ключевая логика плана опирается на твой текущий код: tiered AdaptiveRAGPipeline, hybrid retrieval, session/KB overlay, route mapping и confidence/fallback слой.

В плане также учтены ограничения и возможности Open WebUI: поддержка внешних vector DB, Pipes/Functions для кастомной логики, а также то, что для multi-worker/multi-replica нельзя оставаться на локальном SQLite-backed vector backend.

За основу продуктового retrieval-слоя в ТЗ заложен Qdrant как основной вариант из-за multitenancy и payload filtering, с Milvus как альтернативой для более тяжёлых multi-vector/hybrid сценариев.

Если хочешь, следующим сообщением я могу сделать ещё и вторую версию этого же ТЗ в формате "roadmap по неделям / спринтам".

9. Пользователь

Пишет, что "Этот файл больше недоступен."

10. Ассистент

Пересоздал файл. Вот рабочая ссылка:

[Скачать openwebui_migration_tz.md](sandbox:/mnt/data/openwebui_migration_tz.md)

Если снова будет неудобно, я могу сразу здесь же вставить весь markdown в код-блок.

11. Пользователь

Сформируй pdf файл нашей переписки полностью (без сокращений)